

Decision Control Structures in C++ (if-else, switch, ternary)

[DEF] Definition to Remember

Decision control structures in C++ enable conditional branching using if-else ladders, switch statements with jump tables, and ternary conditional operators.

1. Concept Overview & Fundamentals

Decision Control Structures in C++ (if-else, switch, ternary) provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Decision Control Structure (if-else, switch, goto, ternary)" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: AbsoluteTO

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include
using
```

```
//
int ma
//
in

//
co
//
ci

//
//
in

//
co

//
re
}
```

Problem 2: CategorizeAgeGroup

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include

using

//
int ma
//
in

//
```

```
cc
//
ci

//
if

}
//
el

}
//
el

}
//
el

}
//
re
}
```

Problem 3: GradeCalculator

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 * A -
 * B -
 * C -
 * D -
 * F -
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
```

```
*
* 4.
*
*
*
* ===
*/

#include

// Use
using

//
int ma
do

//
co
//
ci

//
if

//
}

//
}

//
}

//
}

//
re
}
```

Problem 4: GreatestNumber

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
* Que
* Cre
*/

/*
* ===
* CON
```

```
* ===
* 1.
*
*
* 2.
*
*
* 3
*
*
* 4.
*
*
*
*
* ===
*/

#include

// Use
using

//
int ma
in

//
cc
//
ci

//
if

//
}

//
}

//
re
}
```

Problem 5: LeapYear

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
* Que
* Cre
* (cc
*/
```

```
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 *  
 * 2.  
 *  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
  
//  
co  
//  
ci  
  
//  
if  
  
//  
}  
  
}  
  
//  
re  
}
```

Problem 6: MinimumNumberTO

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * CON
```

```
/* ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3
 *
 *
 * 4.
 *
 *
 *
 * ===
 */

#include

// Use
using

//
int ma
in

//
co
//
ci
in

//
co

//
re
}
```

Problem 7: NumberChecker

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
```

```
*
*
* 4.
*
*
*
* ===
*/

#include

// Use
using

//
int ma
in

//
co
//
ci

//
if

//
}

//
}

//
re
}
```

Problem 8: OddEven

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
```

```
 * 3.
 *
 *
 * 4.
 *
 *
 *
 * ===
 */

#include

// Use
using

//
int ma
in

//
co
//
ci

//
if

//
}

}

//
re
}
```

Problem 9: OddEvenTO

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
```

```

* 4.
*
*
*
* ==
*/

#include
#include

// Use
using

//
int main()
{
    //
    // co
    // ci
    // st
    // co
    // re
}

```

Problem 10: PrintMonthSwitch

Problem Description & Objective:

C++ Code Implementation:

Implement

```

/*
 * Que
 * Cre
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

```

```
#inclu
using
//
int ma
//
in
//
co
//
ci
//
sw
}
//
re
}
```

Problem 11: ScoreCategoryTO

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 * usi
 */
```

```
/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 *
 * 2.
 *
 *
 *
 * 3.
 *
 *
 *
 * 4.
 *
 *
 *
 * ===
 */

#include
#include

// Use
using

//
int ma
do
//
co
//
ci
st
//
co
//
re
}
```

Problem 12: SimpleCalculatorSwitch

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 * bas
 */

/*
 * ===
 * CON
 * ===
 * 1.
```

```
*
*
* 2.
*
*
* 3.
*
*
* 4.
*
*
*
*
* ==
*/

#include

// Use
using

//
int ma
ch
do

//
co
//
ci

//
co
//
ci

//
sw
```

```
}  
//  
re  
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. switch statements utilize compiler jump tables for O(1) multi-way branch efficiency.
2. Nested if-else handles complex multi-condition interval checks (e.g. Grade calculations, Leap years).
3. Modern C++ supports if statements with initializer: `if (int x = getVal(); x > 0) { ... }`.
4. break statement terminates switch block immediately; default handles fallback cases.
5. Logical operators (&&, ||) combine relational conditions with short-circuit efficiency.

[RECALL] Quick Recall

Branch Evaluation -> Jump Table Switch -> Ternary Inline -> Scope Bound Initializer